AVIGNON UNIVERSITÉ

Parseur PDF
Rapport de précision

L3 CMI Informatique

Angelo ADRAGNA
Corentin SACCOMAN
Elliot POZUCEK
Colin PALAZZETTI RUBERA
Elvin FICHOT-CORGERON

18/05/2025

Licence d'Informatique CMI
UE conception et qualité logicielles (S-E06-0274) - Semestre 6

Responsables
Juan Manuel TORRES MORENO

UFR
SCIENCES
TECHNOLOGIES
SANTÉ

AVIGNON UNIVERSITÉ
CERI

CENTRE
D'**ENSEIGNEMENT**
ET DE **RECHERCHE**
EN **INFORMATIQUE**
ceri.univ-avignon.fr

Contexte et Objectif

Dans le cadre de notre projet de parseur de fichiers scientifiques, le Département de contrôle qualité du LIA nous a demandé d'évaluer la **précision** du système de détection automatique de sections. Pour cela, une comparaison a été réalisée entre les résultats du parseur et un *corpus de référence* constitué manuellement.

La précision est définie par la formule :

$$\text{Précision} = \frac{\text{Sections correctes trouvées par le système}}{\text{Sections trouvées par le système}}$$

Deux types de frontières sont considérées : strictes (toutes les phrases correctement incluses) et souples (± 2 phrases tolérées).

Méthodologie

1. Répartition des rôles

- **Angelo Adragna** : SCRUM Master, Product Owner, rédaction du rapport et coordination.
- **Corentin Saccoman & Elliot Pozucek** : calculs de précision avec frontières strictes.
- **Colin Palazzetti Rubera & Elvin Fichot-Corgeron** : annotation manuelle du corpus de référence avec les frontières souples.

2. Création du corpus de test

Dix articles scientifiques ont été sélectionnés. Pour chaque article, nous avons :

- Extrait les sections avec le parseur (mode XML).
- Annoté manuellement les véritables frontières dans un fichier `frontieres.xml`.
- Compté le nombre de sections détectées, correctes, et réellement présentes.
- Utilisé le script `precision.py` pour calculer les scores de précision.

Une option `--no-verbose` a été ajoutée pour faciliter les tests.

Résultats

- **Nombre d'articles testés** : 10
- **Précision moyenne observée** : **66.49%**
- **Fichiers avec formats complexes** : baisse significative de la précision (jusqu'à 0%)

```
A_Benders_Decomposition_Approach_toCorrelation_Clustering.pdf                : 100.00% (7/7)
A_memetic_algorithm_for_community_detectionin_signed_networks.pdf            : 28.57% (2/7)
An_Improved_Branch-and-Cut_Code_for_the_Maximum_Balanced_Subgraph_of_a_Signed_Graph.pdf : 66.67% (4/6)
Cabrera_RESUMES_2019.pdf                                                     : 62.50% (5/8)
Conversational_Networks_for_Automatic_Online_Moderation.pdf                  : 85.71% (6/7)
Dynamical_Models_Explaining_Socia_Balance_and_Evolution_of_Cooperation.pdf   : 42.86% (3/7)
Exact_Clustering_via_Integer_Programming_and_Maximum_Satisfiability.pdf      : 71.43% (5/7)
LDA_resume.pdf                                                               : 75.00% (6/8)
Partitioning_large_signed_two-mode_networks_Problems_and_prospects.pdf       : 57.14% (4/7)
Polibits_42_02.pdf                                                           : 75.00% (6/8)

Précision globale : 66.49%
```

Les performances varient fortement selon la qualité du PDF (présence de colonnes, tableaux, etc.). Un cas extrême (article 2) a fortement impacté la moyenne globale.

Analyse et pistes d'amélioration

- **Robustesse** : Le parseur souffre de problèmes liés à la diversité des structures PDF. Des traitements adaptatifs sont nécessaires (ex : détection automatique de colonnes).
- **Débogage facilité** : L'ajout du mode `--no-verbose` et du fichier `precision.py` ont facilité les tests.

Conclusion

Ce sprint a permis d'établir une première évaluation quantitative du système. Malgré des résultats encore perfectibles, la précision moyenne de **67%** montre un comportement correct sur la majorité des cas. L'analyse fine des échecs oriente les améliorations à venir sur la robustesse du pré-traitement et la gestion des formats complexes.

Un sprint supplémentaire aurait logiquement permis une nette amélioration des performances, en intégrant ce corpus de test dans une phase d'apprentissage supervisé, afin d'ajuster dynamiquement les heuristiques de détection aux cas récurrents d'erreur observés.

Lien GitHub : github.com/conception-logicielle-CERI/parseur-pdf